

web application

OWASP Vulnerability SP

Practical testing / attack types

Recommendation / Protection

Broken Access Control

1. **Parameter tampering:** URL/request me id, user_id, account_id jaise identifiers change karke doosre user's object ko access karne ki koshish.
2. **Guess/forced browsing:** /admin, /admin/users, /manage jaise restricted functions ko normal user session se request karna.
3. **UI bypass:** burp se buttons/menus ko ignore karke burp se backend request directly bhejna.
4. **HTTP method tampering:** GET se blocked operation ko POST/PUT/DELETE ke through try karna.
5. **Horizontal privilege test:** User A → User B resource. **6. Vertical privilege test:** normal user → admin function.

1. Broken Access Control

1. Enforce authorization on the server side.
2. Follow a deny-by-default approach.
3. Use a centralized access-control policy.
4. Apply authorization checks to every sensitive object and function.
5. Enable security-event logging and alerting.
6. Implement rate limiting and abuse controls.
7. Invalidate the session when the user logs out.
8. Do not rely on client-side restrictions.

Security Misconfiguration

1. **Verbose error:** invalid request dekar stack trace/version/path disclosure check.
2. **Debug mode:** debug pages/config exposed hain ya nahi.
3. **Default credentials:** test environment me default/admin credentials remain to nahi.
4. **HTTP methods:** unnecessary OPTIONS/PUT/DELETE etc. enabled.
5. **CORS:** overly broad origins/credentials
6. **Security headers:** CSP, HSTS, X-Content-Type-Options etc. review.
7. **Exposed admin/management endpoint.**
8. **Directory listing / backup files.**

2. Security Misconfiguration

1. Maintain a secure baseline/configuration standard.
2. Disable debug mode.
3. Remove or change default accounts and credentials.
4. Disable unnecessary services and features.
5. Restrict management interfaces.
6. Explicitly allowlist CORS origins.
7. Use safe error messages.
8. Implement secure headers and TLS configuration.
9. Perform regular configuration reviews.

1. Authorization server-side enforce karo.
2. Deny-by default follow karo.
3. Centralized access-control policy use karo.
4. Har sensitive object/function par authorization check lagao.
5. Security-event logging + alerting enable karo.
6. Rate limiting/abuse controls lagao.
7. Logout par session invalidate karo.
8. Client-side restrictions par trust mat karo.

1. Secure baseline/configuration standard maintain karo.
2. Debug disable karo.
3. Default accounts/credentials remove/change karo.
4. Unnecessary services/features disable karo.
5. Management interface restrict karo.
6. CORS explicitly allowlist karo.
7. Safe error messages use karo.
8. Secure headers and TLS configuration implement karo.
9. Regular configuration review karo.

Software Supply Chain Failures

1. **Dependency inventory:** libraries/frameworks identify karo.
2. **Version analysis:** old/vulnerable versions identify karo.
3. **Transitive dependency review:** dependency ki dependency bhi check karo.
4. **SBOM verification.**
5. **Package source review:** trusted repository use ho rahi hai ya nahi.
6. **CI/CD review:** build pipeline me unauthorized modification possible hai ya nahi.
7. **Artifact integrity:** build/release artifacts signed/verified hain ya nahi.**

3. Software Supply Chain Failures

1. Maintain an SBOM.
2. Implement SCA scanning.
3. Pin and verify dependencies.
4. Use trusted repositories.
5. Protect the CI/CD pipeline.
6. Sign and verify build/release artifacts.
7. Define a dependency patching SLA.
8. Remove or upgrade unsupported components.

1. SBOM maintain karo.
2. SCA scanning implement karo.
3. Dependencies pin/verify karo.
4. Trusted repositories use karo.
5. CI/CD pipeline protect karo.
6. Build/release artifacts sign & verify karo.
7. Dependency patch SLA define karo.
8. Unsupported components remove/upgrade karo.

Cryptographic Failures

1. **Sensitive data over HTTP:** credentials/session/PII insecure transport par ja raha hai ya nahi.
2. **TLS review:** weak protocol/cipher/certificate configuration.
3. **Cookie security:** Secure/HttpOnly/SameSite flags.
4. **Password storage:** plaintext/weak hashing indication.
5. **Hardcoded secrets/keys:** application code/config me secrets.
- **6. Weak/deprecated algorithm usage.
7. **Sensitive data unnecessarily stored.**

4. Cryptographic Failures

1. Use strong TLS for sensitive traffic.
2. Store passwords using dedicated password-hashing algorithms.
3. Keep cryptographic keys separate from application source code.
4. Use centralized key management.
5. Remove deprecated cryptographic mechanisms.
6. Minimize sensitive data.
7. Configure secure cookie attributes.
8. Maintain a key rotation and revocation process.

1. Sensitive traffic ke liye strong TLS use karo.
2. Passwords ko dedicated password-hashing algorithms se store karo. 3. Keys ko application source se separate rakho.
4. Centralized key management use karo.
5. Deprecated crypto remove karo.
6. Sensitive data minimize karo.
7. Secure cookie attributes configure karo.
8. Key rotation/revocation process rakho.

Injection

1. **SQL injection:** parameters/search/filter inputs ko controlled test values se test karna.
2. **NoSQL injection:** JSON/query operators ke handling test.
3. **OS command injection:** user input command execution context tak pahunchta hai ya nahi.
4. **LDAP injection.**
5. **Template injection.**
6. **Header-based injection.**
7. **XML/XPath injection where applicable.** Testing me baseline request vs modified request compare karo aur backend error/behavior observe karo.

5. Injection

1. Use parameterized queries/prepared statements.
2. Do not concatenate user input into command strings.
3. Use allowlist validation where practical.
4. Apply context-aware output encoding.
5. Use safe framework APIs.
6. Apply least privilege to backend service accounts.
7. Avoid unnecessary interpreter or shell execution

1. Parameterized queries/prepared statements use karo.
2. User input ko command string me concatenate mat karo.
3. Allowlist validation use karo jahan practical ho.
4. Context-aware output encoding karo. 5. Safe framework APIs use karo.
6. Backend service accounts ko least privilege do.
7. Unnecessary interpreters/shell execution avoid karo.

Insecure Design

1. **Business-logic abuse:** workflow ko expected sequence se bahar execute karna.
2. **Trust-boundary testing:** client-side value par excessive trust.
3. **Missing anti-automation controls.**
4. **Missing transaction limits.**
5. **Password-reset/account-recovery design weaknesses.**
6. **Multi-step process me authorization missing.**
7. **Threat-model gaps identify karna.**

6. Insecure Design

1. Perform threat modeling before development.
2. Define abuse cases.
3. Include security requirements in the architecture.
4. Enforce business rules on the server side.
5. Apply transaction limits and risk controls to sensitive workflows.
6. Make security design reviews mandatory.

1. Threat modeling development se pehle karo.
2. Abuse cases define karo.
3. Security requirements architecture me include karo.
4. Server-side business rules enforce karo.
5. Sensitive workflows par transaction limits/risk controls lagao.
6. Security design review mandatory karo.

Authentication Failures

1. **Login bypass attempts.**
2. **Weak password policy.**
3. **Brute-force/rate-limit testing in authorized lab.**
4. **Session fixation.**
5. **Session token reuse after logout.**
6. **Password-reset workflow testing.**
7. **MFA enforcement/flow review.**
8. **Remember-me/session persistence review.**
9. **Reauthentication for sensitive actions.**

7. Authentication Failures

1. Use a strong authentication mechanism.
2. Implement MFA where appropriate.
3. Rate-limit login attempts.
4. Rotate the session after authentication or privilege changes.
5. Invalidate the session/token on logout.
6. Implement secure password recovery.
7. Require reauthentication for sensitive actions.
8. Log and monitor authentication events.

1. Strong authentication mechanism use karo.
2. MFA implement karo where appropriate.
3. Rate-limit login attempts.
4. Session rotate karo after authentication/privilege change.
5. Logout par session/token invalidate karo.
6. Secure password recovery implement karo.
7. Sensitive actions ke liye reauthentication use karo.
8. Authentication events log/monitor karo.

Software or Data Integrity Failures

1. **Insecure update mechanism.**
2. **Unsigned/unverified software artifacts**
3. **CI/CD artifact manipulation possibility.**
4. **Unsafe deserialization paths.**
5. **Client/server data integrity assumptions**
6. **Third-party package/artifact trust review.**

8. Software or Data Integrity Failures

1. Sign software and artifacts.
2. Verify signatures and integrity.
3. Strongly protect CI/CD access.
4. Avoid unsafe deserialization.
5. Maintain build provenance.
6. Make integrity checks mandatory in the deployment pipeline.
7. Apply least privilege.

1. Software/artifacts sign karo.
2. Signature/integrity verify karo.
3. CI/CD access strongly protect karo.
4. Unsafe deserialization avoid karo.
5. Build provenance maintain karo.
6. Deployment pipeline me integrity checks mandatory karo.
7. Least privilege apply karo.

Security Logging and Alerting Failures

1. **Failed login generate karo → log banta hai ya nahi.**
2. **Unauthorized admin access attempt → alert generate hota hai ya nahi.**
3. **Privilege change → audit event verify.**
4. **Password reset/account change → audit trail check.**
5. **Input/security failures → logging verify.**
6. **Log fields:** timestamp, user, source, action, result, correlation ID.
7. **Check whether logs can be modified/deleted by normal users.**

9. Security Logging and Alerting Failures

1. Log security events centrally.
2. Integrate with a SIEM.
3. Create alerts for high-risk events.
4. Keep logs tamper-resistant.
5. Maintain time synchronization.
6. Define a log-retention policy.
7. Link monitoring with incident-response playbooks.
8. Do not unnecessarily store sensitive data in logs.

1. Security events centrally log karo.
2. SIEM integration karo.
3. High-risk events par alerts banao.
4. Logs tamper-resistant rakho.
5. Time synchronization maintain karo.
6. Log retention policy define karo.
7. Monitoring + incident-response playbooks link karo.
8. Sensitive data ko logs me unnecessarily store mat karo.

Mishandling of Exceptional Conditions

- **1. Invalid input/state tests.
2. Missing parameters.
3. Timeout/connection-failure behavior.
4. Unexpected server response.
5. Concurrent request/state handling.
6. Transaction failure halfway through workflow.
7. Error handling me fail-open behavior check. **Goal:** exception ke time security control bypass hota hai ya fail-closed?

10. Mishandling of Exceptional Conditions

1. Follow a fail-closed design.
2. Use centralized exception handling.
3. Do not expose sensitive information in error responses.
4. Ensure transaction rollback and integrity.
5. Handle timeouts properly.
6. Implement concurrency and state controls.
7. Capture exceptional events in security logs.

1. Fail-closed design follow karo.
2. Centralized exception handling use karo.
3. Sensitive information error response me expose mat karo.
4. Transaction rollback/integrity ensure karo.
5. Timeouts properly handle karo.
6. Concurrency/state controls implement karo.
7. Exceptional events ko security logs me capture karo.

OWASP Vulnerability

API1:20 Broken Object Level Authorization (BOLA)

Practical testing / attack types

1. **Parameter tampering:** id, user_id, account_id, order_id change karke doosre user ka object access karna.
2. **Horizontal access:** User A ke token se User B ka object read/update/delete karna.
3. **Object ID enumeration:** sequential/random object IDs ko test karna.
4. **Nested object testing:** /users/101/orders/5001 me parent/child IDs manipulate karna.
5. **HTTP method testing:** same object par GET/PUT/PATCH/DELETE authorization compare karna.
6. **UUID/reference substitution:** valid reference ko another valid reference se replace karna.

API1:2023 — Broken Object Level Authorization (BOLA)

1. Perform server-side authorization checks for every object access.
2. Follow a deny-by-default approach.
3. Perform ownership and tenant validation on the backend.
4. Do not treat a client-supplied ID as proof of authorization.
5. Use a centralized authorization policy.
6. Strictly block cross-tenant access.
7. Log and alert on unauthorized access attempts.

Recommendation / Protection

1. Har object access par server-side authorization check karo.
2. Deny-by-default follow karo.
3. Ownership/tenant validation backend par karo.
4. Client-supplied ID ko authorization ka proof mat samjho.
5. Centralized authorization policy use karo.
6. Cross-tenant access ko strictly block karo.
7. Unauthorized access attempts log + alert karo.

API2:20 Broken Authentication

1. **Missing authentication:** protected endpoint bina token call karna.
2. **Invalid token:** modified/expired/revoked token test karna.
3. **Token validation:** issuer, audience, expiry, signature validation verify karna.
4. **Authentication bypass:** alternative endpoint/version/method se protected function access karna.
5. **Brute-force protection:** authorized lab me repeated login attempts ke baad throttling/lockout check karna.
6. **MFA bypass:** MFA ke baad direct sensitive endpoint access test.
7. **Session/token lifecycle:** logout/revocation ke baad token reuse check.

API2:2023 — Broken Authentication

1. Use a strong authentication mechanism.
2. Validate token signature, issuer, audience, and expiration.
3. Use short-lived access tokens.
4. Securely manage the refresh-token lifecycle.
5. Rate-limit login attempts.
6. Implement MFA where appropriate.
7. Invalidate the session/token upon logout or revocation.
8. Centrally log authentication events.

1. Strong authentication mechanism use karo.
2. Token signature, issuer, audience and expiry validate karo.
3. Short-lived access tokens use karo
4. Refresh-token lifecycle securely manage karo.
5. Login attempts rate-limit karo.
6. MFA where appropriate implement karo.
7. Logout/revocation par session/token invalidate karo.
8. Authentication events centrally log karo.

API3:20 Broken Object Property Level Authorization (BOPLA)

1. **Excessive data exposure:** API response me unnecessary fields mil rahe hain ya nahi.
2. **Hidden/sensitive fields:** isAdmin, role, salary, internal_id, reset_token jaise fields response me check karna.
3. **Mass assignment:** request body me normally allowed fields ke saath additional properties insert karke server behavior dekhna.
4. **Read vs write authorization:** field read allowed hai but update allowed nahi hona chahiye—ye verify karna.
5. **Role-based field testing:** different roles ke response compare karna.

API3:2023 — Broken Object Property Level Authorization (BOPLA)

1. Return only the fields that are required.
2. Use an explicit field allowlist/DTO.
3. Apply field-level authorization to sensitive fields.
4. Prevent mass assignment.
5. Do not allow server-controlled properties to be writable by the client.
6. Appropriately restrict response schemas for different roles.

1. Response me only required fields return karo.
2. Explicit field allowlist/DTO use karo.
3. Sensitive fields ke liye field-level authorization lagao.
4. Mass assignment prevent karo.
5. Server-controlled properties ko client se writable mat rakho.
6. Different roles ke liye response schema appropriately restrict karo.

API4:20 Unrestricted Resource Consumption

1. **Rate-limit testing:** repeated requests controlled lab me bhejkar throttling check karna.
2. **Large payload:** excessive JSON/body size submit karna.
3. **Pagination abuse:** very large limit/page_size values test karna
4. **Expensive operation:** report/search/export jaise resource-heavy APIs test karna.
5. **Concurrent request testing:** controlled parallel requests se resource handling observe karna.
6. **File upload:** size/type/count limits verify karna.
7. **Timeout testing:** slow/long-running requests par limits check karna.

API4:2023 — Unrestricted Resource Consumption

1. Implement per-user, per-IP, and per-token rate limiting.
2. Define request and body size limits.
3. Set a maximum pagination limit.
4. Enforce file upload size and count limits.
5. Implement timeouts and concurrency limits.
6. Define resource quotas.
7. Protect expensive APIs with authorization and throttling.
8. Enable monitoring and alerting for abnormal resource consumption.

1. Per-user/IP/token rate limiting lagao.
2. Request/body size limits define karo.
3. Pagination maximum set karo.
4. File upload size/count limits rakho.
5. Timeouts and concurrency limits implement karo.
6. Resource quotas define karo.
7. Expensive APIs ko authorization + throttling se protect karo.
8. Abnormal consumption par monitoring/alerting enable karo.

API5:20 Broken Function Level Authorization (BFLA)

1. **Admin endpoint guessing:** /admin, /admin/users, /management etc. ko low-privileged token se request karna.
2. **HTTP method tampering:** GET/POST/PUT/PATCH/DELETE ke authorization differences check karna.
3. **Endpoint substitution:** /user/... ko /admin/... pattern se test karna.
4. **Role switching:** normal user vs manager vs admin requests compare karna.
5. **Hidden API testing:** frontend me hidden function ko directly backend se call karna.
6. **Version testing:** old API version me privileged function accessible hai ya nahi.

API5:2023 — Broken Function Level Authorization (BFLA)

1. Enforce function-level authorization on the server side.
2. Centralize RBAC/ABAC.
3. Perform role/permission checks for every privileged function.
4. Do not consider a hidden UI button a security control.
5. Retire old and deprecated endpoints.
6. Log and alert on unauthorized function calls.

1. Function-level authorization server-side enforce karo.
2. RBAC/ABAC centralized rakho.
3. Har privileged function par role/permission check karo.
4. Hidden UI button ko security control mat samjho.
5. Old/deprecated endpoints retire karo.
6. Unauthorized function calls log + alert karo.

API6:20 Unrestricted Access to Sensitive Business Flows

1. **Account creation automation:** repeated creation workflow test.
2. **Login/OTP abuse:** automated repeated attempts ke protection controls check karna.
3. **Password reset abuse:** excessive/reset automation test.
4. **Coupon/promotion workflow:** repeated redemption logic test.
5. **Booking/order workflow:** same business action repeatedly execute karna.
6. **Transaction workflow:** limits, step-up verification and anti-bot controls verify karna.
7. **Workflow bypass:** expected sequence ke steps skip karke next API directly call karna.

API6:2023 — Unrestricted Access to Sensitive Business Flows

1. Identify sensitive business flows.
2. Implement rate limiting and quotas.
3. Implement anti-automation and bot controls.
4. Require step-up verification for risky actions.
5. Define business transaction limits.
6. Validate workflow state on the server side.
7. Implement fraud and risk monitoring.

1. Sensitive business flows identify karo.
2. Rate limiting + quotas use karo.
3. Anti-automation/bot controls implement karo.
4. Step-up verification for risky actions.
5. Business transaction limits define karo.
6. Server-side workflow state validate karo
7. Fraud/risk monitoring implement karo.

API7:20 Server-Side Request Forgery (SSRF)

1. **URL parameter testing:** API jo user-supplied URL fetch karti hai us input ko controlled lab endpoint se test karna.
2. **Webhook testing:** callback/webhook destination validation check karna.
3. **Import/fetch function:** remote resource import feature test karna.
4. **Redirect handling:** server redirects ko follow karta hai ya nahi.
5. **Protocol validation:** HTTP/HTTPS ke alawa unexpected schemes accept hote hain ya nahi.
6. **Internal destination protection:** lab network me internal-only service ko application se reachable hai ya nahi.

PI7:2023 — Server-Side Request Forgery (SSRF)

1. Use a destination allowlist.
2. Do not blindly fetch user-supplied URLs.
3. Restrict outbound network egress.
4. Block internal/private destinations where applicable.
5. Strictly validate redirect handling.
6. Restrict allowed schemes and ports.
7. Provide network isolation for SSRF-prone services.

1. Destination allowlist use karo.
2. User-supplied URLs ko blindly fetch mat karo.
3. Outbound network egress restrict karo.
4. Internal/private destinations block karo where applicable.
5. Redirect handling strictly validate karo.
6. Allowed schemes/ports restrict karo.
7. SSRF-prone service ko network isolation do.

API8:20 Security Misconfiguration

1. **Verbose errors:** invalid API request bhejkar stack trace/version/path disclosure check.
2. **CORS review:** allowed origins/credentials verify.
3. **Debug endpoints:** Swagger/debug/management interfaces exposed hain ya nahi
4. **Unnecessary methods:** OPTIONS/PUT/DELETE etc. unnecessarily enabled hain ya nahi.
5. **Default settings/credentials review.**
6. **Security headers/TLS configuration review.**
7. **Old API versions or test endpoints exposed.**

API8:2023 — Security Misconfiguration

1. Maintain a hardened API configuration baseline.
2. Disable debug mode.
3. Remove or change default credentials.
4. Explicitly allowlist CORS.
5. Disable unused endpoints and HTTP methods.
6. Restrict management interfaces.
7. Use safe error responses.
8. Implement API gateway and TLS hardening.

1. Hardened API configuration baseline maintain karo.
2. Debug mode disable karo.
3. Default credentials remove/change karo.
4. CORS explicitly allowlist karo.
5. Unused endpoints/methods disable karo.
6. Management interfaces restrict karo.
7. Safe error responses use karo.
8. API gateway + TLS hardening implement karo.

API9:20 Improper Inventory Management

1. **API discovery:** documented API vs actually deployed API compare karna.
2. **Version enumeration:** /v1, /v2, /v3 etc. check karna.
3. **Shadow APIs:** undocumented endpoints identify karna.
4. **Deprecated API testing:** old versions still accessible hain ya nahi.
5. **Swagger/OpenAPI exposure:** documentation publicly

API9:2023 — Improper Inventory Management

1. Maintain a complete API inventory.
2. Use an API gateway/catalog.
3. Define an API version lifecycle.
4. Formally retire deprecated APIs.
5. Continuously discover shadow APIs.
6. Isolate development and test environments from production.

1. Complete API inventory maintain karo.
2. API gateway/catalog use karo.
3. Version lifecycle define karo.
4. Deprecated APIs formally retire karo.
5. Shadow APIs continuously discover karo.
6. Dev/test environments production se isolate karo.
7. API owner, data type and authentication model

		exposed hai ya nahi. 6. Host discovery: dev/test/staging API accidentally internet-facing hai ya nahi. 7. Endpoint ownership review: every API ka owner/data classification available hai ya nahi.	7. Document the API owner, data type, and authentication model.	document karo.
API10:2	Unsafe Consumption of APIs	1. Third-party API trust review: external response ko automatically trusted maana ja raha hai ya nahi. 2. Response validation: malformed/unexpected fields ke handling ko controlled test environment me verify karna. 3. Schema mismatch: expected datatype ke against unexpected response test 4. Redirect handling: third-party service ke redirects safely handled hain ya nahi. 5. TLS/certificate validation review. 6. Timeout/error handling: external API unavailable ho to application safe behavior rakhti hai ya nahi. 7. External API data injection: third-party returned data downstream interpreters me unsafe way se use ho rahi hai ya nahi.	API10:2023 — Unsafe Consumption of APIs 1. Treat third-party API responses as untrusted input. 2. Implement strict schema and data validation. 3. Enforce TLS and certificate verification. 4. Define timeouts and retry limits. 5. Apply least privilege to external integrations. 6. Handle third-party API failures safely. 7. Apply appropriate validation and encoding before passing data to downstream SQL, HTML, command, or template sinks. 8. Continuously monitor third-party dependencies and integrations.	1. Third-party API responses ko untrusted input treat karo .2. Strict schema/data validation implement karo. 3. TLS/certificate verification enforce karo. 4. Timeouts and retry limits define karo. 5. External integrations ko least privilege do. 6. Third-party API failures safely handle karo. 7. Downstream SQL/HTML/command/template sinks se pehle appropriate validation/encoding karo. 8. Third-party dependencies and integrations continuously monitor karo.
From https://chatgpt.com/g/8a6fca44-3c24-83ee-ae03-797023d9f9a6				
LLM VAPT				
API VAPT mein ek important distinction API testing mein API1, API3 aur API5 ko alag rakhna bahut important hai: Category Main Question API1 – BOLA Kya main kisi doosre user's object ko access kar sakta hoon? API3 – BOPLA Kya main object ke unauthorized properties/fields read ya modify kar sakta hoon? API5 – BFLA Kya main aisa function/API operation chala sakta hoon jo meri role ke liye allowed nahi hai?				
LLM VAPT				
OWASP	Vulnerability	Practical testing / attack types 1. Direct prompt injection: user prompt ke through model ke intended behavior/instructions ko alter karne ki koshish. 2. Instruction override: system/developer instructions ko ignore karne ke liye conflicting instructions dena. 3. Context manipulation: prompt mein fake roles, priorities ya authority introduce karna. 4. Indirect prompt injection: RAG/document/web content mein malicious instructions embed karke model se unhe follow karwane ki koshish. 5. Multi-turn injection: multiple conversations ke through model behavior gradually manipulate karna. 6. Tool-oriented injection: model ko unintended tool/function call karne ke liye influence karna.		
LLM01:2	Prompt Injection		1. Prompt Injection 1. Logically separate user input from trusted instructions. 2. Apply least privilege to tool access. 3. Do not use model output as an authorization mechanism. 4. Treat external and RAG content as untrusted. 5. Apply independent policy validation to tool calls. 6. Require human approval or confirmation for high-impact actions. 7. Implement prompt-injection detection and monitoring.	Recommendation / Protection 1. User input aur trusted instructions ko logically separate rakho. 2. Least-privilege tool access do. 3. Model output ko authorization mechanism mat banao. 4. External/RAG content ko untrusted treat karo. 5. Tool calls par independent policy validation lagao. 6. High-impact actions ke liye human approval/confirmation use karo. 7. Prompt-injection detection + monitoring implement karo.
LLM02:2	Sensitive Information Disclosure	1. Sensitive-data prompting: model se confidential information reveal karne ki controlled lab testing. 2. Context leakage: current conversation/context mein doosre user's confidential data exposure check. 3. RAG leakage: restricted documents retrieve ho rahe hain ya nahi. 4. PII exposure: names, email, financial, credentials, internal data output mein appear ho raha hai ya nahi. 5. Memory/session isolation: ek session/user ka information doosre context mein accessible hai ya nahi. 6. Error/debug disclosure: backend errors, API responses ya traces mein secrets leak ho rahe hain ya nahi.	2. Sensitive Information Disclosure 1. Apply data classification and minimization. 2. Do not unnecessarily send sensitive data into the model context. 3. Enforce access control for RAG. 4. Use PII and secret detection and redaction. 5. Enforce tenant and session isolation. 6. Do not hardcode secrets in prompts or system context. 7. Implement output DLP controls.	1. Data classification + minimization karo. 2. Sensitive data ko model context mein unnecessary mat bhejo. 3. RAG access-control enforce karo. 4. PII/secret detection and redaction use karo. 5. Tenant/session isolation enforce karo. 6. Secrets ko prompts/system context mein hardcode mat karo. 7. Output DLP controls implement karo.
LLM03:2	Supply Chain	1. Model provenance: model/source verify karo. 2. Third-party model/plugin/SDK inventory. 3. Dependency vulnerability review. 4. Model/package integrity: downloaded model/artifact tampered to nahi. 5. Dataset/source trust review. 6. Build/deployment pipeline security. 7. Dependency substitution/typosquatting risks review.	3. Supply Chain 1. Use trusted model repositories. 2. Verify model and artifact integrity. 3. Maintain an SBOM and dependency inventory. 4. Evaluate third-party models and plugins. 5. Protect the CI/CD pipeline. 6. Pin and verify dependencies. 7. Maintain model provenance and version tracking.	1. Trusted model repositories use karo. 2. Model/artifact integrity verify karo. 3. SBOM/dependency inventory maintain karo. 4. Third-party models/plugins evaluate karo. 5. CI/CD pipeline protect karo. 6. Dependencies pin/verify karo. 7. Model provenance aur version tracking maintain karo.
LLM04:2	Data and Model Poisoning	1. Training-data integrity: unauthorized/malicious records identify karo. 2. Fine-tuning dataset review: malicious samples/instructions detect karo. 3. RAG data poisoning: indexed documents mein manipulated content check karo. 4. Embedding poisoning: malicious content retrieval ranking ko influence kar raha hai ya nahi. 5. Backdoor behavior: specific trigger/input par abnormal model behavior test karo. 6. Data-source trust: external ingestion pipeline verify karo.	4. Data and Model Poisoning 1. Maintain provenance for training and fine-tuning data. 2. Validate and sanitize datasets. 3. Use trusted data sources. 4. Implement data integrity, signing, and versioning. 5. Secure the RAG ingestion pipeline. 6. Review and approve dataset changes. 7. Maintain model-behavior baselines and regression testing.	1. Training/fine-tuning data provenance maintain karo. 2. Dataset validation and sanitization karo. 3. Trusted sources use karo. 4. Data integrity/signing/versioning implement karo. 5. RAG ingestion pipeline secure karo. 6. Dataset changes review/approve karo. 7. Model behavior baseline + regression testing maintain karo.
LLM05:2	Improper Output Handling	1. XSS-style testing: model output directly HTML/UI mein render ho raha hai ya nahi. 2. SQL/command sink: LLM-generated output backend query/command mein directly pass ho raha hai ya nahi. 3. Code execution: generated code automatically execute to nahi hota. 4. URI/output injection: model-generated URLs/markup blindly trusted to nahi. 5. Structured output manipulation: JSON/function arguments validate ho rahe hain ya nahi. 6. Markdown/HTML rendering: unsafe content browser/client mein interpreted to nahi.	5. Improper Output Handling 1. Treat LLM output as untrusted input. 2. Apply context-specific encoding and sanitization. 3. Use parameterization for SQL and command inputs. 4. Do not automatically execute generated code. 5. Validate structured output against a schema. 6. Independently authorize and policy-check tool arguments.	1. LLM output ko untrusted input treat karo. 2. Context-specific encoding/sanitization karo.3. SQL/command inputs ke liye parameterization use karo. 4. Generated code ko automatically execute mat karo. 5. Structured output schema validate karo. 6. Tool arguments ko independent authorization/policy checks se validate karo.
LLM06:2	Excessive Agency	1. Tool inventory: model kin tools/functions ko call kar sakta hai. 2. Excessive permissions: model ko required se zyada privileges diye gaye hain ya nahi. 3. Autonomous action: model sensitive action human approval ke bina execute kar sakta hai ya nahi. 4. Chained actions: ek low-risk request se multiple high-impact tool calls possible hain ya nahi. 5. Parameter manipulation: generated tool arguments insufficiently checked hain ya nahi. 6. Cross-system access: model unnecessary external systems/resources access kar sakta hai ya nahi.	6. Excessive Agency 1. Apply least privilege. 2. Maintain a tool/function allowlist. 3. Require human approval for high-risk actions. 4. Independently authorize every tool call. 5. Validate tool arguments against a schema. 6. Limit the blast radius. 7. Enable tool-execution logging and monitoring.	1. Least privilege apply karo. 2. Tool/function allowlist maintain karo. 3. High-risk actions ke liye human approval rakho. 4. Every tool call ko independently authorize karo. 5. Tool arguments schema-validate karo. 6. Blast radius limit karo. 7. Tool execution logging + monitoring enable karo.
LLM07:2	System Prompt Leakage	1. System-instruction disclosure testing: model se hidden instructions disclose karne ke attempts.	7. System Prompt Leakage 1. Do not treat the system prompt as a secret-storage	1. System prompt ko secret-storage mechanism mat samjho.

	1. Indirect extraction: conversation/context manipulation ke through hidden instructions reveal hote hain ya nahi. 3. Role/format tricks: output formatting ke through hidden configuration expose hote hai ya nahi. 4. Error-based disclosure: malformed requests se system/developer instructions leak hote hain ya nahi. 5. Secret placement review: API keys/passwords/confidential policy system prompt mein stored hain ya nahi.	mechanism. 2. Do not store secrets in prompts. 3. Enforce sensitive authorization logic in backend services. 4. Ensure prompt disclosure does not create catastrophic impact. 5. Separate sensitive configuration from the model context.	7. Secrets prompt mein store mat karo. 3. Sensitive authorization logic backend services mein enforce karo. 4. Prompt disclosure ko catastrophic impact na banne do. 5. Sensitive configuration ko model context se separate rakho.
LLM08:2 025	Vector and Embedding Weaknesses 1. RAG authorization test: User A restricted document ko retrieve kar sakta hai ya nahi. 2. Cross-tenant retrieval: tenant A ke query results mein tenant B data appear hota hai ya nahi. 3. Document poisoning: malicious/incorrect indexed content retrieval influence kar raha hai ya nahi. 4. Metadata manipulation: document metadata/ACLs tamperable hain ya nahi. 5. Embedding collision/similarity issues: unintended documents retrieval mein aa rahe hain ya nahi. 6. Retrieval boundary testing: top-k/filters ke through restricted content expose to nahi.	8. Vector and Embedding Weaknesses 1. Enforce authorization at retrieval time. 2. Protect document-level ACL metadata. 3. Enforce tenant isolation. 4. Validate and sanitize ingested content. 5. Use a trusted embedding pipeline. 6. Protect metadata integrity. 7. Filter retrieval results according to user permissions.	1. Retrieval-time authorization enforce karo. 2. Document-level ACL metadata protect karo. 3. Tenant isolation enforce karo. 4. Ingested content validate/sanitize karo. 5. Trusted embedding pipeline use karo. 6. Metadata integrity protect karo. 7. Retrieval results ko user permissions ke against filter karo.
LLM09:2 025	Misinformation 1. Factuality testing: known-answer questions ke against model response compare karo. 2. Hallucination testing: model unsupported facts invent karta hai ya nahi. 3. RAG grounding: answer retrieved source se actually supported hai ya nahi. 4. Citation verification: model citations/source references valid hain ya nahi. 5. Confidence testing: incorrect answer ko excessive certainty ke saath present karta hai ya nahi. 6. High-impact domain testing: sensitive decision workflows mein unsupported output use ho raha hai ya nahi.	9. Misinformation 1. Use grounding/RAG where appropriate. 2. Implement source verification. 3. Require human review for high-impact decisions. 4. Communicate confidence and uncertainty appropriately. 5. Maintain automated factuality and regression tests. 6. Do not use model output as the sole authoritative source.	1. Grounding/RAG use karo where appropriate. 2. Source verification implement karo. 3. High-impact decisions ke liye human review rakho. 4. Confidence/uncertainty appropriately communicate karo. 5. Automated factuality/regression tests maintain karo. 6. Model output ko sole authoritative source mat banao.
LLM10:2 025	Unbounded Consumption 1. Large-input testing: excessive prompt/context size handling check. 2. Large-output testing: maximum output/token limits verify karo. 3. Repeated request testing: controlled lab mein rate limiting/throttling check karo. 4. Expensive operation testing: repeated tool/model calls ka cost/resource impact check karo. 5. Recursive/agent loop: workflow uncontrolled repeated calls kar sakta hai ya nahi. 6. File/context processing limits: very large uploads/context windows handle kaise hote hain. 7. Quota bypass: user/IP/API key limits properly enforce ho rahe hain ya nahi.	10. Unbounded Consumption 1. Set input/token limits. 2. Set output/token limits. 3. Implement per-user/API-key rate limiting. 4. Define model/tool call budgets. 5. Set maximum iterations for agent loops. 6. Set file/context size limits. 7. Implement cost/resource monitoring and alerting. 8. Use quotas and circuit breakers.	1. Input/token limits set karo. 2. Output/token limits set karo. 3. Per-user/API-key rate limiting implement karo. 4. Model/tool call budgets define karo. 5. Agent loops ke maximum iterations rakho. 6. File/context size limits rakho. 7. Cost/resource monitoring + alerting implement karo. 8. Quotas and circuit breakers use karo.

From <https://chatgpt.com/s/8afbc6e432d483ba-acc2-797d21f0eead>

OWA SP	Vulnerability	Practical testing / attack types	Recommendation / Protection
M1:2 024	Improper Credential Usage	1. Hardcoded credentials: APK/IPA, resources, configs aur source/decompiled code mein API keys, passwords, tokens, certificates search karo. 2. Insecure credential storage: Android SharedPreferences/files ya iOS UserDefaults/plain files mein credentials stored hain ya nahi. 3. Embedded secrets: production app mein reusable service/API secrets present hain ya nahi. 4. Credential leakage through logs: login/token-related values Android Logcat ya iOS logs mein aa rahe hain ya nahi. 5. Weak token handling: long-lived access tokens improperly stored/reused ho rahe hain ya nahi. 6. Backup/extraction test: app data backup/extraction ke baad sensitive credentials accessible hain ya nahi.	1. Long-lived secrets app ke andar hardcoded mat karo. 2. Android Keystore / iOS Keychain use karo. 3. Short-lived access tokens use karo. 4. Refresh-token lifecycle securely manage karo. 5. Logs mein credentials/tokens mat likho. 6. Compromised credentials ko revoke/rotate karne ka mechanism rakho. 7. Backend secrets ko server-side rakho.
M2:2 024	Insecure Authentication/Authorization	1. Authentication bypass: protected screen/API ko authentication ke bina invoke karne ki koshish. 2. Token manipulation: invalid/expired session token ke behavior ko test karo. 3. Session reuse: logout ke baad old session/token valid hai ya nahi 4. Client-side authorization: app sirf UI hide karke admin function protect to nahi kar rahi. 5. Role testing: normal user aur privileged user ke requests compare karo. 6. Biometric flow: biometric success/failure ke baad sensitive action ka server-side authorization check verify karo. 7. Deep-link authentication: protected deep link ko unauthenticated state mein open karke behavior dekho.	1. Authentication + authorization server-side enforce karo. 2. Client ko trust boundary mat samjho. 3. Strong session/token lifecycle maintain karo. 4. Logout/revocation properly implement karo. 5. Sensitive actions ke liye reauthentication/step-up authentication use karo. 6. Biometric ko authorization ka sole replacement na banao; secure platform APIs use karo.
M3:2 024	Insecure Communication	1. Cleartext traffic: HTTP endpoints identify karo. 2. TLS validation: certificate/hostname validation properly ho rahi hai ya nahi. 3. Custom trust logic: insecure custom certificate validation/trust managers inspect karo. 4. Mixed-content communication: secure app ka koi request insecure channel par fallback karta hai ya nahi. 5. Sensitive traffic inspection: credentials, tokens, PII encrypted transport se ja rahe hain ya nahi. Android: Network Security Configuration review iOS: ATS configuration aur networking implementation review.	1. HTTPS/TLS use karo. 2. Certificate and hostname validation properly enforce karo. 3. Cleartext traffic disable karo unless explicitly required. 4. Insecure custom trust managers avoid karo. 5. Sensitive API communications ko authenticated encrypted channel par rakho. 6. Development exceptions ko production build mein disable karo.
M4:2 024	Insecure Data Storage	1. Local database: SQLite/Core Data files mein PII, tokens, passwords etc. check karo. 2. Preferences: Android SharedPreferences / iOS UserDefaults inspect karo. 3. Files/cache: app sandbox mein sensitive files/cache identify karo. 4. Logs: Android Logcat/iOS logs inspect karo. 5. Screenshots: sensitive screens screenshot/app-switcher preview mein expose ho rahe hain ya nahi. 6. Backups: backup/restore behavior se sensitive data recover hota hai ya nahi. 7. Clipboard: sensitive values unnecessarily clipboard mein ja rahe hain ya nahi.	1. Sensitive data ko minimum rakho. 2. Android Keystore-backed encryption aur iOS Keychain use karo. 3. Sensitive local databases/files appropriately encrypt/protect karo. 4. Logs mein secrets/PII mat store karo. 5. Secure backup policy configure karo. 6. Sensitive data ki retention minimize karo. 7. Clipboard aur screenshots ko risk ke hisaab se control karo.
M5:2 024	Insufficient Cryptography	1. Weak algorithm: deprecated/weak crypto implementation identify karo. 2. Hardcoded encryption keys: binary/source/config mein keys search karo. 3. Weak random generation: security-sensitive tokens/keys ke liye predictable random functions use ho rahe hain ya nahi.	1. Platform cryptographic APIs use karo. 2. Strong, approved algorithms use karo. 3. Keys ko secure hardware-backed/platform storage mein rakho where appropriate.
		2. Insecure Authentication/Authorization 1. Enforce authentication and authorization on the server side. 2. Do not treat the client as a trusted boundary. 3. Maintain a strong session/token lifecycle. 4. Properly implement logout and revocation. 5. Use reauthentication/step-up authentication for sensitive actions. 6. Do not use biometrics as the sole authorization mechanism; use secure platform APIs.	
		3. Insecure Communication 1. Use HTTPS/TLS. 2. Properly enforce certificate and hostname validation. 3. Disable cleartext traffic unless explicitly required. 4. Avoid insecure custom trust managers. 5. Keep sensitive API communication on authenticated encrypted channels. 6. Disable development exceptions in production builds.	
		4. Insecure Data Storage 1. Minimize sensitive data storage. 2. Use Android Keystore-backed encryption and iOS Keychain. 3. Properly encrypt/protect sensitive local databases and files. 4. Do not store secrets or PII in logs. 5. Configure a secure backup policy. 6. Minimize sensitive-data retention. 7. Control clipboard usage and screenshots according to risk.	

		4. Static IV/nonce: encryption implementation mein reuse/misconfiguration check karo. 5. Key storage: keys app files/preferences mein plain stored to nahi. 6. Custom crypto implementation: home-grown encryption identify karo. 7. Key lifecycle: generation, storage, rotation/revocation review.	where appropriate. 4. Do not hardcode cryptographic keys. 5. Use a secure random number generator. 6. Implement nonce/IV handling correctly. 7. Avoid home-grown cryptography.	4. Cryptographic keys hardcode mat karo. 5. Secure random generator use karo. 6. Nonce/IV handling correctly implement karo. 7. Home-grown cryptography avoid karo.
M6:2 024	Insecure Code Quality	1. Input validation review. 2. Unsafe API usage: deprecated/insecure platform APIs identify karo. 3. Memory/resource handling: risky native-code patterns where applicable 4. Error handling: sensitive information exception/error mein leak to nahi 5. WebView code: unsafe JavaScript/interface handling inspect karo. 6. Native bridge: Android JNI / iOS native interfaces ke trust boundaries inspect karo. 7. Static-analysis findings: high-risk code patterns identify karo.	6. Insecure Code Quality 1. Adopt secure coding standards. 2. Integrate SAST/static analysis. 3. Replace dangerous or deprecated APIs. 4. Use input validation and safe output handling. 5. Tightly restrict WebView and native bridges. 6. Include code review and security testing in CI/CD.	1. Secure coding standards adopt karo. 2. SAST/static analysis integrate karo. 3. Dangerous/deprecated APIs replace karo. 4. Input validation + safe output handling use karo. 5. WebView/native bridges tightly restrict karo. 6. Code review + security testing CI/CD mein include karo.
M7:2 024	Insufficient Binary Protections	1. Debuggable build: production app debug-enabled hai ya nahi. 2. Binary analysis: package ko decompile karke sensitive logic/strings identify karo. 3. Reverse-engineering resistance: critical client-side logic unnecessarily exposed hai ya nahi. 4. Code signing: application properly signed hai ya nahi. 5. Runtime integrity: modified/tampered application ke against security controls review karo. 6. Sensitive logic location: security-critical decisions entirely client-side hain ya nahi.	7. Insufficient Binary Protections 1. Keep production builds debug-disabled. 2. Use strong application signing. 3. Keep sensitive business/security decisions on the server side. 4. Use obfuscation as defense-in-depth where justified. 5. Implement integrity/tamper protections according to risk. 6. Do not store secrets in the client binary.	1. Production builds ko debug-disabled rakho. 2. Strong application signing use karo. 3. Sensitive business/security decisions server-side rakho. 4. Obfuscation ko defense-in-depth ke roop mein use karo where justified. 5. Integrity/tamper protections risk ke according implement karo. 6. Secrets ko client binary mein mat rakho.
M8:2 024	Security Misconfiguration	1. Android Manifest / iOS entitlements review. 2. Exported components: Android Activities, Services, Receivers, Providers unnecessarily exported hain ya nahi. 3. Excessive permissions: app unnecessarily high-risk permissions request kar rahi hai ya nahi. 4. Debug configuration: debug flags/test endpoints enabled hain ya nahi. 5. Backup configuration: app data backup policy secure hai ya nahi. 6. WebView settings: unnecessary JavaScript/file access enabled hai ya nahi. 7. iOS URL schemes/deep links/entitlements: overly broad or unsafe configuration.	8. Security Misconfiguration 1. Use least-privilege permissions. 2. Export Android components only when required. 3. Remove debug/test configurations from production. 4. Securely configure the backup policy. 5. Harden WebViews. 6. Keep iOS entitlements to the minimum required scope. 7. Maintain and review a configuration baseline.	1. Least-privilege permissions use karo. 2. Android components ko required hone par hi exported rakho. 3. Debug/test configuration production se remove karo. 4. Backup policy securely configure karo. 5. WebView harden karo. 6. iOS entitlements ko minimum required scope tak rakho. 7. Configuration baseline maintain + review karo.
M9:2 024	Insecure Data Storage —	Note: Current OWASP Mobile Top 10 2024 mein M9 specifically Insecure Data Storage nahi hai; M9 = Improper Session Handling. Older Mobile Top 10 versions ke saath confusion hota hai.	9. Insecure Data Storage 1. Minimize sensitive local data. 2. Use secure platform storage such as Android Keystore and iOS Keychain. 3. Encrypt/protect sensitive databases and files. 4. Protect logs from sensitive-data exposure. 5. Secure backup and restore mechanisms. 6. Minimize data retention.	Session security ko dedicated control ke roop mein test karo: secure token storage, expiration, revocation, logout invalidation, reauthentication aur session binding. (OWASP Developer Guide)
M9:2 024	Insecure Session Handling	1. Logout test: logout ke baad old token/session reuse. 2. Session timeout: inactivity ke baad session remains valid ya expire hota hai. 3. Token rotation: login/privilege change ke baad token rotate hota hai ya nahi. 4. Concurrent sessions: old devices/sessions ko revoke karne ka mechanism hai ya nahi. 5. Device change: sensitive session new/untrusted device context mein unnecessarily valid to nahi. 6. Biometric unlock: local unlock ko server-side session authorization se incorrectly equate to nahi kiya gaya.		1. Short-lived sessions use karo. 2. Logout par tokens/session invalidate karo. 3. Sensitive events par session/token rotate karo. 4. Refresh tokens securely protect karo. 5. Device/session management provide karo where needed. 6. Sensitive actions par reauthentication/step-up authentication use karo.
M10:2 024	Insufficient Security Controls	1. Runtime abuse controls review. 2. Anti-tamper/integrity controls where justified. 3. Root/jailbreak risk handling review. 4. Debugger/runtime instrumentation exposure assessment in authorized test builds. 5. Fraud/abuse controls: sensitive functionality ko automated misuse se protect kiya gaya hai ya nahi. 6. Paid/protected functionality: client-side checks alone par depend to nahi. 7. Defense-in-depth review: authentication, authorization, integrity, monitoring sab independent layers mein hain ya nahi.	10. Insufficient Cryptography 1. Use platform cryptographic APIs. 2. Use strong, approved algorithms. 3. Store cryptographic keys securely. 4. Do not hardcode keys. 5. Use secure random number generation. 6. Implement correct IV/nonce handling. 7. Avoid custom cryptographic implementations.	1. Security controls ko layered/defense-in-depth approach mein implement karo. 2. Critical operations server-side enforce karo. 3. Runtime integrity controls risk-based rakho. 4. Abuse/fraud monitoring implement karo. 5. Sensitive functionality ko client-only checks se protect mat karo. 6. Security events monitor/log karo. 7. Controls ko regular security testing se validate karo.

From <<https://owasp.org/Top10/A-2024-03-top-10-2024.html>>

Q4. What is Rules of Engagement?

Answer:

Rules of Engagement define **what you are authorized to test and how you are allowed to test it.**

It normally includes:

- IP/domain/application scope
- Testing dates
- Allowed techniques
- Out-of-scope systems
- Production restrictions
- DoS restrictions
- Social engineering restrictions
- Data-handling requirements
- Emergency contact
- Reporting process

Q6. What is reconnaissance?

Reconnaissance means collecting information about the authorized target.

Passive

- Public DNS information
- Certificate information
- Publicly available technology information
- Public attack-surface information

Active

- Port/service discovery
- Application enumeration
- Endpoint discovery
- API enumeration

Interview line:

"I keep reconnaissance strictly within the approved scope."

Q25. What do you check in Android VAPT?

Major areas:

Application

- Authentication
- Authorization
- Local storage
- Cryptography
- Network communication
- SSL/TLS
- Certificate validation
- Exported components
- Intents
- Deep links
- WebViews
- Clipboard
- Logs
- Backup
- Debug configuration

Q26. Static vs Dynamic mobile analysis?

Static

Analyze application without executing it.

Examples:

- APK structure
- Manifest
- Decompiled code
- Permissions
- Hardcoded secrets
- Crypto implementation

Dynamic

Application is executed and behavior is monitored.

Examples:

- Network traffic
- Runtime behavior
- API calls
- Local storage
- Runtime instrumentation

Q33. What network vulnerabilities would you look for?

Examples:

- Weak protocols
- Unnecessary exposed services
- Weak authentication
- Outdated software
- Misconfiguration
- Insecure management interfaces
- Weak TLS configuration
- Excessive network exposure
- Default credentials
- Segmentation issues

Q34. What do you check during cloud security assessment?

Think in categories:

IAM

- Excessive privileges
- Weak policies
- Privilege escalation paths

Storage

- Public exposure
- Excessive permissions
- Sensitive information

Network

- Security groups
- Internet exposure
- Segmentation

Compute

- Vulnerable images

Q36. What do you check during wireless assessment?

High-level areas:

- Encryption configuration
- Authentication
- WPA/WPA2/WPA3 configuration
- Rogue access points
- Weak configurations
- Guest network isolation
- Client isolation
- Management interface security
- Segmentation
- Enterprise authentication configuration

- Management interfaces
- Metadata exposure
- Logging**
 - Audit logging
 - Monitoring
 - Alerting
- Secrets**
 - Keys
 - Tokens
 - Credentials

Q39. How do you prioritize vulnerabilities?

I consider:

Technical Severity
+
Exploitability
+
Internet Exposure
+
Asset Criticality
+
Data Sensitivity
+
Business Impact
+
Existing Controls

Q40. What should a professional VAPT report contain?

A strong report includes:

Executive Summary
For management.
Scope
What was tested.
Methodology
How testing was performed.
Risk Summary
Example:

Critical: 1
High: 5
Medium: 8
Low: 10
Detailed Findings
Each finding:

Title
Severity
CVSS
Affected Asset
Description
Technical Details
Evidence
Impact
Root Cause
Remediation
References
Retest Status

Q44. What do you check during retest?

Three things:

1. Original vulnerability is fixed
2. Exploitation path no longer works
3. Fix didn't introduce another vulnerability

You are given a customer portal with Web UI + REST API + Android application. You have 5 days to perform VAPT. Explain your complete approach."

Ideal answer:

DAY 1
Scope & Architecture
↓
Reconnaissance
↓
Attack Surface Mapping
↓
Web/API/Mobile Enumeration

DAY 2
Authentication
Authorization
Session Management
API Security
Input Validation
DAY 3
Business Logic
Mobile Security
Configuration
Network/Infrastructure
Cloud Exposure
DAY 4
Manual Validation
Risk Assessment
Evidence Collection
Finding Correlation
DAY 5
Report
Client Discussion
Remediation Recommendations
Retest Planning

PART 1 — ANDROID PENTESTING: 10 QUESTIONS & ANSWERS

Q1. How do you perform Android application penetration testing?

Answer:

"I follow a combination of static and dynamic analysis. I start by understanding the application scope, APK/AAB, application architecture and backend APIs. During static analysis I inspect the AndroidManifest, permissions, exported components, hardcoded secrets, cryptography and application code. During dynamic analysis I observe runtime behavior, network communication, local storage, authentication, authorization and security controls. I also test the backend APIs independently because mobile application security depends heavily on server-side controls."

Typical workflow:

APK
↓
Static Analysis
↓
Manifest / Code / Resources
↓
Dynamic Analysis
↓
Runtime + Network + Storage
↓
API Testing
↓
Risk Assessment
↓
Report
↓
Retest

Q2. What do you check in AndroidManifest.xml?

Answer:

I specifically check:

- Permissions
- Exported activities
- Exported services
- Exported broadcast receivers
- Content providers
- Intent filters
- Deep links
- Debuggable configuration
- Backup configuration
- Network security configuration
- SDK versions
- Components exposed unnecessarily

For example:

If a sensitive activity is exported without appropriate authorization controls, another application may potentially invoke it.

Interview tip: Don't say "exported = vulnerability."

An exported component can be legitimate; you need to assess whether its exposure creates a security issue.

Q3. What is an exported Android component?

Answer:

An exported component is an Android component that can potentially be invoked by another application or external mechanism, depending on its configuration.

Components include:

PART 2 — iOS PENTESTING: 10 QUESTIONS & ANSWERS

Q1. How is iOS application penetration testing different from Android?

Answer:

"The overall security objectives are similar, but the platform architecture and security mechanisms are different. On iOS I pay particular attention to the application sandbox, entitlements, Keychain, URL schemes, App Transport Security, data protection, IPC mechanisms and native APIs."

Also test:

iOS App
↓
Local Storage
↓
Keychain
↓
Network
↓
Authentication
↓
API
↓
Backend

Q2. What is the iOS application sandbox?

Answer:

The sandbox isolates an application's files and resources from other applications.

During testing I assess whether sensitive information is:

- Stored insecurely
- Accessible through unintended mechanisms
- Exposed through application functionality
- Improperly protected through backups or other data-sharing mechanisms

Important interview point:

A sandbox provides isolation, but it does not automatically make insecure application design secure.

Q3. What is Keychain and why is it important in iOS pentesting?

Answer:

Keychain is Apple's secure storage mechanism commonly used for sensitive credentials and secrets.

I check:

- What sensitive information is stored
- Whether Keychain is used appropriately
- Accessibility/protection configuration
- Whether secrets unnecessarily exist elsewhere
- Whether authentication/session information is handled securely

Example:

Credentials
↓
Keychain
↓
Application
Rather than:

Credentials
↓
Plaintext file ❌

"Give me the difference between Android and iOS mobile pentesting."

Use this answer:

Area	Android	iOS
App package	APK/AAB	IPA/App bundle
Main config	AndroidManifest.xml	Info.plist
Secure storage	Keystore + app storage	Keychain
Isolation	Application sandbox	Application sandbox
Components	Activities, Services, Receivers, Providers	App lifecycle / extensions / URL handling etc.
Deep linking	Intents / App Links	URL Schemes / Universal Links
Network controls	Network Security Configuration	ATS
Runtime analysis	Frida etc.	Frida etc.
Static analysis	DEX/native code/resources	Mach-O/native code/resources
Permissions	Android permissions	iOS permissions/entitlements
Backend /API	Critical	Critical

10. Practical Logic-Flaw Checklist

Mobile/API/Web VAPT mein main ye checklist use karunga:

Authentication

- Password reset abuse
- OTP reuse
- OTP workflow manipulation
- Verification workflow bypass
- Account recovery logic

Authorization

- Horizontal privilege escalation
- Vertical privilege escalation
- Object ownership
- Function-level authorization

Transactions

- Duplicate transaction
- Replay
- Race condition
- Double spending
- Negative/invalid quantities
- Transaction state manipulation

Coupons / Rewards

Q3. What are exposed external components?

Answer:

An exported component is an Android component that can potentially be invoked by another application or external mechanism, depending on its configuration.

Components include:

- Activity
- Service
- Broadcast Receiver
- Content Provider

During pen testing I check:

Whether the component is intentionally exposed and whether proper permission and authorization controls are enforced.

Q4. What sensitive data do you look for in Android local storage?

Answer:

I inspect locations such as:

- SharedPreferences
- SQLite databases
- Application files
- Cache
- External storage
- Logs
- WebView storage
- Local configuration files

I look for:

- Authentication tokens
- Session information
- Personal information
- API keys
- Credentials
- Cryptographic keys
- Sensitive application data

The important question is:

Is sensitive data stored unnecessarily, and if it must be stored, is it adequately protected?

Q5. How would you test for hardcoded secrets in an APK?

Answer:

During static analysis I inspect:

- Decompiled code
- Resources
- Configuration files
- String constants
- Build artifacts
- Native libraries where applicable

I look for:

API keys
Tokens
Passwords
Private keys
Encryption keys
Internal endpoints
Cloud credentials

Then I **validate whether the discovered value is actually sensitive and usable**, rather than reporting every string that looks like a secret.

Q6. What is Android SSL/TLS certificate pinning?

Answer:

Certificate pinning adds an additional validation mechanism where the application expects a particular certificate or public-key identity rather than relying solely on the device's normal trusted CA chain.

During an authorized assessment, I determine:

1. Whether pinning exists.
2. Where validation occurs.
3. Whether sensitive traffic is properly protected.
4. Whether the implementation can be bypassed in the test environment.
5. Whether sensitive information becomes observable during testing.

Q7. What is Frida and how can it help in Android VAPT?

Answer:

"Frida is a dynamic instrumentation framework that allows security testers to observe and modify application behavior at runtime."

In Android VAPT it can help with:

- Runtime method observation
- Understanding application behavior
- Security-control validation
- Certificate-validation analysis
- Runtime instrumentation
- Debugging/research

Interview follow-up may be:

"Why would you use Frida instead of only static analysis?"

Answer:

Static analysis tells me what the application contains; dynamic instrumentation helps me understand what actually happens at runtime.

Q8. What Android WebView vulnerabilities do you check?

Answer:

I check:

- Unsafe JavaScript configuration
- JavaScript interfaces
- Untrusted URL loading
- Insecure navigation
- File access settings
- Mixed content
- Improper input handling
- Sensitive data exposure
- WebView-to-native interaction

Particularly:

I assess whether untrusted web content can interact with privileged native functionality.

Q9. How do you test Android deep links?

Answer:

I identify:

- Custom URL schemes
- Android App Links
- Intent filters
- Deep-link activities

Then verify:

- Whether sensitive activities can be invoked externally
- Whether authentication is required
- Whether authorization is enforced
- Whether parameters are validated
- Whether malicious/untrusted links can trigger sensitive functionality

Appropriate:

Rather than:

Credentials



Plaintext file ✗

Q4. What is ATS in iOS?

ATS = **App Transport Security**.

It is a set of networking security requirements intended to encourage secure communication.

During testing I check whether:

- HTTPS is used appropriately
- Weak/insecure connections are permitted
- Exceptions are configured unnecessarily
- TLS configuration is appropriate
- Certificate validation is correctly implemented

Q5. What are iOS entitlements?

Answer:

Entitlements are security-related capabilities associated with an application.

They can control access to platform capabilities and services.

During an assessment I inspect entitlements to understand:

- Application capabilities
- Associated domains
- Keychain-related access
- Push notifications
- App groups
- Other privileged capabilities

Interview line:

"I don't treat the presence of an entitlement itself as a vulnerability; I assess whether the capability is required and securely implemented."

Q6. What are URL schemes in iOS?

URL schemes allow an application to respond to specially formatted URLs.

Example concept:

myapp://something

During testing I check whether:

- Sensitive functionality can be triggered
- Authentication is required
- Authorization is enforced
- Parameters are validated
- Sensitive information can be passed through URLs
- Another application could potentially invoke the functionality unexpectedly

Q7. What is iOS Universal Links?

Answer:

Universal Links allow web URLs to open associated functionality inside an iOS application.

During testing I check:

- Domain association
- Link handling
- Authentication
- Authorization
- Parameter validation
- Sensitive functionality exposed through links

The security principle is the same:

Never assume that because a function is invoked through a trusted-looking link, it is automatically authorized.

Q8. How do you perform static analysis of an iOS application?

Answer:

Depending on the authorized assessment environment, I examine:

- IPA/application bundle
- Info.plist
- Entitlements
- Embedded frameworks
- Binary characteristics
- Strings
- Configuration
- URL schemes
- API endpoints
- Cryptographic implementation
- Sensitive data handling

I then correlate static findings with runtime behavior.

Q9. How do you test SSL certificate validation/pinning in iOS?

Answer:

"First I identify how the application performs TLS certificate validation. I then determine whether certificate pinning is implemented and validate its effectiveness in an authorized test environment. I observe whether sensitive traffic can be intercepted when the application's expected security controls are challenged."

Then assess:

- TLS configuration
- Certificate validation
- Pinning implementation
- Sensitive data exposure
- Error handling

Q10. What are the major areas you check during iOS VAPT?

Strong answer:

"I assess authentication, authorization, secure storage, Keychain usage, cryptography, network communication, ATS configuration, certificate validation, URL schemes, Universal Links, WebViews, IPC mechanisms, data protection, logging, application configuration, reverse-engineering resistance and backend APIs."

Appropriately

- Race condition
- Double spending
- Negative/invalid quantities
- Transaction state manipulation

Coupons / Rewards

- Coupon reuse
- Coupon stacking
- Expired coupon acceptance
- Unauthorized coupon application
- Reward duplication

Business Workflow

Step 1 → Step 2 → Step 3 → Step 4

Test:

- Can Step 3 be directly called?
- Can Step 2 be repeated?
- Can Step 4 happen before Step 3?
- Can requests be skipped?
- Can parameters be changed between steps?

1. Logic Flaw Testing kya hota hai?

Technical vulnerability mein application ka code/control galat ho sakta hai.

Logic flaw mein application **business process ko galat tarike se implement** karti hai.

Example:

Normal workflow:

Apply Coupon



Validate Coupon



Apply Discount



Mark Coupon Used

Agar attacker same coupon ko multiple times valid kara sake:

Request 1 → Validate → Apply

Request 2 → Validate → Apply

Request 3 → Validate → Apply



Coupon used only once
but discount applied 3 times

Ye **business logic flaw / race condition** ho sakta hai.

2. Race Condition kya hai?

Race condition tab hoti hai jab application ke do ya more concurrent requests ke beech **timing/order** ki wajah se security control fail ho jata hai.

Classic example:

Account balance:

Balance = ₹1000

Application ka logic:

Check balance >= ₹1000



Debit ₹1000

Agar do requests simultaneously processing mein aa jayein:

Request A → Check ₹1000 available → YES

Request B → Check ₹1000 available → YES

Request A → Debit ₹1000

Request B → Debit ₹1000

Agar backend atomic transaction use nahi karta, application inconsistent state mein ja sakti hai.

3. Practical Lab Example — Coupon

Suppose test application mein:

Coupon: TEST100

Discount: ₹100

Allowed usage: 1

Normal test

Burp mein request capture karo:

```
POST /api/coupon/apply
Authorization: Bearer <test-token>
{
  "coupon":"TEST100"
}
```

Pehli request:

200 OK

Discount applied

Second request:

400

Coupon already used

So far, good.

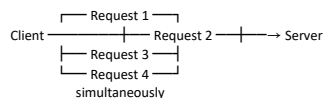
4. Race Condition Test

Ab interesting part.

Goal:

Kya multiple requests ek hi time par process hone par server coupon ko ek se zyada baar accept karta hai?

Conceptually:



Expected:

1 request → SUCCESS

others → REJECTED

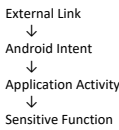
Vulnerable behavior:

Request 1 → SUCCESS

Then verify:

- Whether sensitive activities can be invoked externally
- Whether authentication is required
- Whether authorization is enforced
- Whether parameters are validated
- Whether malicious/untrusted links can trigger sensitive functionality

Example concept:



Q10. What are the most important Android VAPT areas according to you?
Strong interview answer:

"I would focus on insecure local storage, insecure cryptography, authentication and authorization, insecure network communication, certificate validation, exported components, WebViews, deep links, hardcoded secrets, improper platform usage, debugging/configuration issues and backend API security."

And finish with:

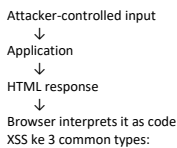
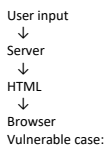
"I would also map findings to the applicable OWASP Mobile Application Security Verification Standard and OWASP Mobile Top 10 categories."

🔥 VERY IMPORTANT: ANDROID vs iOS INTERVIEW COMPARISON
The interviewer may suddenly ask:

1. XSS actually hota kya hai?

Application user input ko browser mein **unsafe way se render** kar deti hai.

Normal:



1. **Reflected XSS**
2. **Stored XSS**
3. **DOM-based XSS**

2. Reflected XSS — Practical

Suppose application mein search function hai:

/search?q=hello
Browser mein response:

Search results for: hello
Step 1 — Input identify karo
Burp Proxy se request capture:

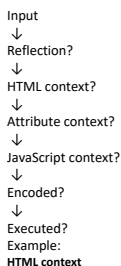
GET /search?q=hello HTTP/1.1
Host: lab.example
Step 2 — Harmless proof-of-concept input
Authorized lab mein test input:

<script>alert(1)</script>
Request conceptually:

GET /search?q=<script>alert(1)</script>
Agar application input ko HTML context mein unsafe tarike se reflect karti hai aur browser JavaScript execute karta hai, **reflected XSS confirmed**.

3. Lekin sirf <script> test karna enough nahi

Modern applications mein filtering/WAF ho sakta hai.
Aapko pehle determine karna hai:



<div>USER_INPUT</div>
Attribute context

<input value="USER_INPUT">
JavaScript context

<script>
var x = "USER_INPUT";
</script>

Context determine karna XSS testing ka core concept hai.

4. Burp Suite se practical workflow

Simple language mein: CORS browser ka ek **security mechanism** hai jo decide karta hai ki **ek origin ki web page, doosre origin ke server response ko browser mein read kar sakti hai ya nahi**.

1. Pehle "Origin" samjho

Origin =

Scheme + Host + Port

Example:

<https://bank.com>
aur

<https://evil.com>

ye **different origins** hain.

2. CORS ka basic scenario

Maan lo user bank.com par logged in hai.

Ab evil.com ki webpage browser se request bhejne ki koshish karti hai:

evil.com

↓

request

↓

bank.com/api/account

Browser ka same-origin policy normally doosre origin ko response **read** karne se restrict karta hai.

CORS server ko controlled way mein specify karne deta hai:

"Kis origin ko mera response read karne ki permission hai?"

3. CORS Header

Server response mein:

Access-Control-Allow-Origin: <https://trusted.com>

iska matlab:

<https://trusted.com> ko response read karne ki permission hai.

4. Dangerous CORS configuration

Suppose sensitive API:

<https://bank.com/api/profile>

Response:

Access-Control-Allow-Origin: <https://evil.com>

Agar authentication credentials bhi cross-origin request ke saath appropriately allowed/configured hain, to ye potentially serious CORS misconfiguration ho sakti hai.

Important: Sirf Access-Control-Allow-Origin dekhkar vulnerability declare nahi karni.

5. Credentials wala important concept

CORS testing mein ye header bahut important hai:

Access-Control-Allow-Credentials: true

Typical risky combination ko assess karte waqt:

Access-Control-Allow-Origin: <https://attacker-controlled-origin>

Access-Control-Allow-Credentials: true

Phir determine karna hota hai:

- User authenticated hai?
- Authentication cookies use ho rahi hain?
- Cookies SameSite policy se protected hain?
- Sensitive response browser mein actually readable hai?
- Server origin dynamically reflect kar raha hai?
- Preflight properly configured hai?

6. Practical VAPT testing

Burp Suite mein API request/response capture karo.

Suppose:

GET /api/profile HTTP/1.1
Host: target.example
Origin: <https://test.example>
Cookie: session=...
Response:

HTTP/1.1 200 OK

Access-Control-Allow-Origin: <https://test.example>

Ab assess karo ki server arbitrary origins accept karta hai ya nahi.

Test conceptually different origins ke saath:

Origin: <https://trusted.example>
Origin: <https://untrusted.example>
Origin: <https://attacker-controlled.example>
Compare:

Request Origin

↓

Server Response

↓

Access-Control-Allow-Origin

↓

Allow-Credentials?

↓

Can sensitive response actually be read?

7. Wildcard * ka kya matlab hai?

Agar response hai:

Access-Control-Allow-Origin: *
iska matlab broadly:

Any origin may access the response **subject to browser CORS rules**.

Lekin ek common interview trap:

*** + credentials automatically vulnerable nahi hai.**

Browsers credentialed CORS requests ke saath wildcard ko restrict karte hain. Isiliye actual exploitability validate karni hoti hai.

8. CORS vs CSRF

Ye dono confuse mat karna.

CSRF

Attacker victim ke browser se unwanted action karwane ki koshish karta hai.

1 request → SUCCESS

others → REJECTED

Vulnerable behavior:

Request 1 → SUCCESS

Request 2 → SUCCESS

Request 3 → SUCCESS

provided the coupon was intended for one-time use.

5. Burp Suite mein practical approach

Authorized lab mein:

Step 1

Request capture karo.

Step 2

Request ko **Repeater** mein bhejo.

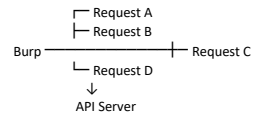
Step 3

Same request ki multiple copies prepare karo.

Step 4

Burp ke supported concurrent-request functionality ka use karke requests ko as close to simultaneously as possible send karo.

Concept:



Step 5

Responses compare karo:

Request Status Discount Result

A	200	₹100	Success
B	200	₹100	Success
C	400	₹0	Rejected
D	400	₹0	Rejected

Agar **sirf ek success** hai → likely properly protected.

Agar multiple successful applications milti hain → investigate race condition.

6. Sirf HTTP 200 dekhna enough nahi hai

Ye interview mein bahut important point hai.

Suppose:

Request A → 200

Request B → 200

Request C → 200

Isse alone vulnerability prove nahi hoti.

Aapko verify karna hai:

Database state

+

Account balance

+

Order state

+

Coupon usage

+

Actual business impact

For example API 200 return kar sakti hai but backend actually sirf ek transaction commit kar raha ho.

7. Race Condition ke common test cases

A. Coupon reuse

One-time coupon

↓

Concurrent requests

↓

Multiple successful discounts?

B. Wallet / balance

₹1000 balance

Request A → ₹1000 withdrawal

Request B → ₹1000 withdrawal

Check whether both transactions actually succeed.

C. Gift card

Gift Card = ₹500

Redeem A

Redeem B

Redeem C

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

↓

```
var x = "USER_INPUT";
</script>
```

Context determine karna XSS testing ka core concept hai.

4. Burp Suite se practical workflow

Browser
↓
Burp Proxy
↓
Capture Request
↓
Send to Repeater
↓
Modify Parameter
↓
Send
↓
Inspect Response
↓
Determine Reflection/Context
↓
Browser Validation
Repeater mein aap parameter modify karke compare kar sakte ho.
Example:

```
GET /search?q=TEST123
Response mein:
```

Search results for: TEST123
Toh input reflected hai.
Ab check karo ki application us input ko properly HTML-encode karti hai ya nahi.

5. Stored XSS — Practical

Suppose application mein comment feature hai:

Name:
Comment:
[Submit]
Normal:

Comment = Nice article
Server:

Database
↓
Nice article
Later:

User opens page
↓
Database value retrieved
↓
Browser
Test
Authorized lab mein harmless PoC input submit karo:

```
<script>alert(1)</script>
Agar submit karne ke baad page reload/open karne par script
execute hoti hai:
```

Input
↓
Database
↓
Application
↓
Browser
↓
JavaScript execution
That's **stored XSS**.

6. Reflected vs Stored

	Reflected	Stored
Input stored?	No	Yes
Trigger	Usually crafted request	Later viewing affected content
Persistence	No	Yes
Common location	Search/error pages	Comments/profile/posts
Testing	Request → Response	Submit → Store → Retrieve

7. DOM-Based XSS — Practical Concept

DOM XSS mein server-side reflection necessarily required nahi hoti.
Example vulnerable JavaScript:

```
document.getElementById("output").innerHTML =
  location.hash.substring(1);
URL:
```

<https://lab.example/#hello>

JavaScript browser ke location.hash ko directly innerHTML mein insert kar raha hai.
Flow:

URL Fragment
↓
JavaScript
↓
DOM
↓
innerHTML
↓
Browser interprets content
Yahan server response mein malicious input reflect hona zaroori nahi.

8. XSS identify karte waqt kya evidence collect karoge?

Professional VAPT report mein:
Finding

8. CORS vs CSRF

Ye dono confuse mat karna.

CSRF

Attacker victim ke browser se unwanted action karwane ki koshish karta hai.

Victim browser
↓
Authenticated request
↓
Target application
CORS
Main question hai:

Kya doosre origin ki webpage response ko browser mein read kar sakti hai?

Short trick:
CSRF = action
CORS = cross-origin response access

9. Interview answer

Agar interviewer pooche:

"How do you test CORS misconfiguration?"

Aap bolo:

"I inspect the application's CORS response headers and test how the server handles different Origin values. I check whether arbitrary or untrusted origins are allowed, whether credentials are permitted, whether sensitive authenticated responses are exposed, and whether browser enforcement actually makes the issue exploitable. I then assess the affected data and business impact rather than reporting a CORS header alone as a vulnerability."

Ye senior VAPT-level answer hai.

VAPT mein commonly **double-spend / duplicate transaction / replay / duplicate request** type logic flaws ko refer kiya jata hai.
Example:

```
POST /api/transfer
{
  "amount":1000,
  "to":"TEST_ACCOUNT"
}
```

Normal behavior:

```
₹5000
↓
Transfer ₹1000
↓
₹4000
```

Testing question:

If the exact same legitimate transaction is submitted twice, does the application correctly recognize it as a duplicate, or does it process it twice?

Ye **replay/duplicate transaction logic** ka test ho sakta hai.

9. Race Condition vs Replay Attack

Dono ko confuse mat karna.

Replay / Duplicate Request

Same request ko dobara send karna:

Request
↓
Request again
↓
Request again
Question:
Kya duplicate transaction accept hoti hai?

Race Condition

Requests ko **concurrently / overlapping execution window** mein process karwana:

```
┌─ Request A
├─ Request B
└─ Request C
↓
Concurrent processing
```

Question:

Kya simultaneous processing se validation/control bypass hota hai?

Reflected Cross-Site Scripting

Affected endpoint

/search

Parameter

q

Evidence

Request

↓

Input

↓

Response reflection

↓

Browser execution

Impact

Potential impact depends on the context and application:

- User interaction through malicious content
- Unauthorized actions in victim's browser context
- Exposure of accessible application data
- Phishing/UI manipulation
- Session/account compromise in some application architectures

Impact ko automatically "account takeover" mat likhna.

Actual exploitability prove karni hoti hai.

9. XSS severity kaise decide karoge?

Suppose:

Case 1

XSS only affects attacker's own session.

Potentially lower impact.

Case 2

Stored XSS executes for other authenticated users.

Higher risk.

Case 3

Stored XSS executes in an administrative interface.

Potentially much higher impact because the victim has privileged access.

So severity depends on:

Exploitability

+

Who can trigger it

+

Who is affected

+

Execution context

+

Privileges

+

Data/functionality accessible

+

Business impact

10. Interview mein practical question

Interviewer:

"You found a search parameter reflecting your input. How will you test whether it is XSS?"

Strong answer:

"First I would determine whether my input is reflected and identify the output context, such as HTML, attribute or JavaScript context. I would then check whether the application properly encodes or sanitizes the input. In the authorized test environment, I would use a harmless proof-of-concept to verify whether browser-side script execution is possible. I would then determine whether the issue is reflected, stored or DOM-based, assess the affected users and business impact, and document reproducible evidence."

Ye answer **sirf payload yaad karne se kaafi better** hai.

11. XSS testing ka golden rule

Reflection ≠ XSS

Ye distinction interview mein zaroor bolna.

Input reflected

↓

Is it encoded?

↓

What context?

↓

Can browser interpret it as markup/script?

↓

Can execution actually occur?

↓

What is the impact?

Agar application input ko:

<script>...

mein encode kar deti hai, browser usse text ki tarah treat karega, executable markup ki tarah nahi.